byType1" >}}
  {{< tableau/filters "GROUPLABEL"="dynamic analysis" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://10az.online.tableau.com//site/gitlab/views/DRAFTFlakytestissues/MonthlyFlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="dynamic analysis" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://10az.online.tableau.com//site/gitlab/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="dynamic analysis" >}}
{{< /tableau >}}

 Targets

For our Merge Request types, we have an initial soft target ratio of 60% features, 30% maintenance, and 10% bugs based on the cross-functional prioritization process. This is not a hard target and we expect to see variation in this ratio as we mature and our focus evolves.

SECTION: engineering/development/sec/secure/products/_index.md

---
SECTION: engineering/development/sec/secure/products/metrics.md

This page shows various metrics for the products developed and maintained by
the Secure Stage.

We are actively supporting Common Weakness Enumeration (CWE)
as a standard vulnerability classification system and a common language to discuss software weaknesses.

Using CWE as a foundation has several advantages:

1. CWE is a comprehensive and well-documented system and can be considered as a de-facto standard for discussing software weaknesses.
1. CWE provides mappings to other vulnerability and classification systems and/or rankins (such as OWASP Top 10).
1. CWE provides a stable ontology: definitions can be added but existing definitions do not change (unlike OWASP rankings).

CWE is a hierarchical system with an ontology that is organized in a tree
structure where a parent CWE is more general than its child; a child CWE
captures a vulnerability in more specific terms than its parent.

In contrast to CWE, OWASP Top 10 provides a risk
ranking of the most critical security vulnerabilities. The 10 risk categories
change on a regular basis.

The table below shows the mapping between OWASP categories and their CWE
counterparts. Note that the table includes transitive CWE mappings which
are all the CWE mappings that are listed on the OWASP Top10 website
including their child-CWEs.

OWASP	CWEs
A1: Broken Access Control	8, 9, 13, 15, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34,
35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58,
59, 61, 62, 64, 65, 66, 67, 69, 72, 73, 94, 95, 96, 97, 98, 114, 134, 178, 200, 201, 202, 203,
204, 205, 206, 207, 208, 209, 210, 211, 213, 214, 215, 219, 220, 250, 256, 257, 258, 259, 260,
261, 262, 263, 264, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 281,
282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 293, 294, 295, 296, 297, 298, 299, 300, 301,
302, 303, 304, 305, 306, 307, 308, 309, 312, 313, 314, 315, 316, 317, 318, 321, 322, 346, 350,
352, 359, 370, 374, 375, 377, 378, 379, 386, 402, 403, 419, 420, 421, 422, 424, 425, 426, 427,
428, 433, 441, 470, 472, 488, 491, 492, 493, 497, 498, 499, 500, 502, 520, 521, 522, 523, 524,
525, 526, 527, 528, 529, 530, 531, 532, 535, 536, 537, 538, 539, 540, 541, 548, 549, 550, 551,
552, 553, 555, 556, 565, 566, 582, 583, 593, 598, 599, 601, 603, 608, 612, 615, 619, 620, 621,
623, 627, 638, 639, 640, 642, 645, 647, 648, 651, 668, 706, 708, 732, 767, 784, 798, 804, 827,
836, 842, 862, 863, 913, 914, 915, 918, 921, 922, 923, 925, 926, 927, 939, 940, 941, 942, 1004,
1021, 1022, 1189, 1191, 1220, 1222, 1224, 1230, 1231, 1242, 1243, 1244, 1252, 1254, 1255, 1256,
1257, 1258, 1259, 1260, 1262, 1263, 1267, 1268, 1270, 1273, 1274, 1275, 1276, 1280, 1282, 1283,
1290, 1292, 1294, 1295, 1296, 1297, 1299, 1300, 1302, 1303, 1304, 1311, 1312, 1313, 1314, 1315,
1316, 1317, 1320, 1321, 1323, 1324, 1327, 1328, 1334, 1336|
A2: Cryptographic Failures	5, 6, 259, 261, 296, 310, 319, 321, 322, 323, 324, 325, 326,
327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 347,
523, 587, 720, 757, 759, 760, 780, 798, 804, 818, 916, 1204, 1240, 1241|
A3: Injection	15, 20, 37, 42, 43, 45, 46, 49, 50, 52, 53, 54, 56, 73, 74, 75, 76, 77, 78,
79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 93, 94, 95, 96, 97, 98, 99, 100, 102, 103,
104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 116, 117, 119, 120, 121, 122, 123, 124,
125, 126, 127, 129, 130, 134, 138, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151,
152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 170, 179,
180, 181, 184, 190, 291, 384, 415, 416, 441, 462, 464, 466, 470, 471, 472, 473, 554, 564, 601,
606, 607, 610, 611, 621, 622, 624, 626, 627, 641, 643, 644, 652, 680, 692, 694, 781, 785, 786,
787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 805, 806, 822, 823, 824, 825, 838, 914,
917, 918, 943, 1021, 1173, 1174, 1236, 1284, 1285, 1286, 1287, 1288, 1289, 1336|
A4: Insecure Design	5, 9, 13, 15, 73, 102, 105, 106, 108, 109, 114, 183, 209, 210, 211,
213, 235, 250, 256, 257, 258, 259, 260, 266, 267, 268, 269, 270, 271, 272, 273, 274, 280, 302,
307, 308, 309, 311, 312, 313, 314, 315, 316, 317, 318, 319, 321, 350, 419, 424, 425, 426, 430,
434, 444, 447, 451, 455, 472, 501, 520, 522, 523, 525, 535, 536, 537, 539, 549, 550, 554, 555,
556, 565, 579, 598, 602, 603, 614, 623, 636, 637, 638, 642, 646, 648, 650, 653, 654, 655, 656,
657, 671, 784, 798, 799, 807, 837, 840, 841, 927, 942, 1007, 1021, 1022, 1173, 1174, 1192,
1331|
A5: Security Misconfiguration	2, 7, 11, 12, 13, 15, 16, 258, 260, 315, 520, 526, 537, 541,
547, 555, 611, 614, 756, 776, 942, 1004, 1032, 1174|
A6: Vulnerable and Outdated Components	937, 1035, 1104

A7: Identification and Authentication Failures	13, 255, 256, 257, 258, 259, 260, 261, 262, 263, 287, 288, 289, 290, 291, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 321, 346, 350, 370, 384, 425, 521, 522, 523, 549, 555, 593, 599, 603, 613, 620, 640, 645, 798, 804, 836, 940, 1216, 1299, 1324
A8: Software and Data Integrity Failures	98, 345, 346, 347, 348, 349, 351, 352, 353, 354, 360, 422, 426, 494, 502, 565, 616, 646, 649, 784, 827, 829, 830, 915, 924, 1293, 1321
A9: Security Logging and Monitoring Failures	117, 223, 532, 778
A10: Server-Side Request Forgery (SSRF)/)	918

Below you can find the OWASP and CWE coverage for different secure products.
All charts that are displayed below are powered by live anonymized
vulnerability data from our security scans. These are vulnerabilities we are
actively identifying in real-world customer usage of our security scanning
tools.

OWASP Top 10 2021 Coverage

The chart below depicts the CWEs that map to the OWASP Top 10 2021.
All of these CWEs are detected by GitLab's SAST/DAST
and Dependency Scanning capabilities.

Data unavailable in Tableau

CWE Coverage

SAST

The table below shows the combined
Common Weakness Enumerator (CWE) findings reported by
our SAST analyzers on
projects hosted on gitlab.com

Data unavailable in Tableau

Below you can find a list of which CWEs are detected by each analyzer:

{{% details summary="eslint" %}}

{{% /details %}}

{{% details summary="flawfinder" %}}

{{% /details %}}

{{% details summary="gosec" %}}

{{% /details %}}

{{% details summary="nodejs-scan" %}}

{{% /details %}}

{{% details summary="semgrep" %}}

{{% /details %}}

{{% details summary="spotbugs" %}}

{{% /details %}}

DAST

The table below shows the combined
Common Weakness Enumerator (CWE) findings reported by
our DAST analyzers on
projects hosted on gitlab.com

Data unavailable in Tableau

GitLab Advisory Database for Dependency Scanning

Statistical information about advisories for dependency scanning is available on the GitLab Landing Page for Dependency Scanning Advisories.

SECTION: engineering/development/sec/secure/secret-detection/_index.md

Secret Detection

The Secret Detection group maintains the Secret Detection feature category for customer software repositories.

Common Links

Main Slack channel: gast-secret-detection
Stand-up updates: gast-secret-detection-standup
Slack aliases: @gastsecretdetection

Secret Detection Shared Calendar

The Secret Detection Shared Calendar is used to make sure PTO events are visible to everyone on the team.

Below are the steps to add the calendar to Time Off by Deel:

In Slack, jump to Time Off by Deel > Home, click on the dropdown Your Events, and select Calendar Sync.
Under Additional calendars to include?, click on Add calendar.
Add the following calendar ID:

cb4fda90478cfc15d4ec5fa18952c0c976d3078df887cc3548f8d6592d22de032@group.calendar.google.com.
Great job! · Your PTO events will be synced to Static Anaylsis Shared Calendar from now on. ·

Our Team

{{< team-by-manager-role role="Engineering(.)Manager(.)Secure:Secret Detection" team="Engineer" >}}

How We Work

The Secret Detection group is largely aligned with GitLab's Product Development Flow, however there are some notable differences in
how we seek to deliver software. The engineering team predominantly concerns itself with the delivery of software, which is the portion of the workflow states where
we deviate the most. What follows is how we manage the handoff from product management to engineering to deliver software.

Issues worked by this team can span analyzers, vendored templates, and GitLab's Rails monolith.

Issue Boards

Secret Detection Delivery Board - Primary board for engineers, columns are workflow labels.
Secret Detection Planning Board - Milestone-centric board primarily used by product management to gauge work in current and upcoming milestones.
Secret Detection EM Board - Engineer-centric board used by engineering management to gauge how heavy a load engineer is carrying.
Secret Detection Bug Scrub Board- Bug board, columns are severity.

Issue and Merge Requests labels

GitLab has a labeling convention for issues and Merge Requests. We follow this convention, though there are specific labels required to route artifacts to us. We
use these labels to filter issues meant for us on our issue boards. They are also used for metrics and KPI reporting.

Label	Meaning
~section::sec	Identifies the issue or MR as belonging to the Sec Section's roadmap.
~devops::application security testing	Identifies the issue or MR as belonging to the Secure Stage's roadmap.
~group::secret detection	Identifies the Secret Detection group as the collection of individuals who will work on the issue or MR.
~Category:Secret Detection	Identifies the issue or MR as being part of the Secret Detection feature category.
~backend	Identifies the issue or MR as being part of GitLab's backend.
~frontend	Identifies the issue or MR as being part of GitLab's frontend.

Refinement

We have recently experimented with a new refinement process when refining the issues for the

Secret Push Protection Beta epic. This process draws a lot of inspiration from other sections/stages but also aligns with the current Secure Engineering Refinement.

Following a set of discussions and feedback of said process, we have decided to make the improved refinement process a part of our software delivery workflow.

The goal of the process is to:

Clarify any outstanding questions or concerns.
Add a proposal or an implementation plan.
Determine if the issue is the smallest iteration possible, and break it down if not.
Determine if the issue requires support from other teams.
Assign a weight to the issue.
Ensure the issue is labeled correctly.
Ensure issue is marked as ready to be worked on.

Workflow

The refinement process doesn't concern itself with how issues are picked up to be refined. This is assumed to be done in an earlier process that triages issues from the backlog (whether via the MoSCoW process or a similar variant). Normally, before issues are refined, a planning issue is created to select which issues are picked up in an upcoming milestone to be refined and delivered. This could possibly be done in the Looking Forward section of the planning issue.

This workflow can summarized as follows:

1. Triaging: issues from backlog are triaged to determine the must/should/would/could-haves.
2. Planning/Prioritization: issues are selected for a specific milestone.
3. Refinement: issues are refined and prepared to be picked up.

Steps

Below is a list of steps followed during the refinement process.

The refinement process is kicked off when a planning issue is finalized.
A bot or an automated script assigns a number of issues (e.g. 2-3) randomly to each engineer.
An engineer is responsible for refining their assigned issues, but could ask for help if needed.
Engineers would follow a certain checklist to determine if an issue is refined and ready to be picked up.
The refinement process is time-boxed (e.g. one week), after which all issues ready for development is picked up.
When an engineer completes refining an issue, they pass it on to another engineer (a reviewer) for review.
The reviewer should follow the guidelines outlined in the checklist as much as possible:
  If the reviewer agrees with the engineer, the issue is marked as ready for development.
  If they are in disagreement, they should discuss the reason and find a way forward.
  If a disagreement cannot be resolved, the issue is brought to next team meeting for discussion.
Pending issues can continue to be refined, and depending on their status they may or may not

be included in the milestone.

Checklist

The following checklist is to be copied either in the issue description or posted as a comment in the issue being refined. This is used to clarify the refinement and refinement review progress for all interested stakeholders.

markdown
Please copy the list below into the issue you are refining, and check them as you deem appropriate.

Refinement Progress

If a checkbox is not relevant for the issue, please remove or strikethrough it.

- [ ] This issue describes a problem to solve, or a task to complete, and it's confirmed.
- [ ] This issue describes a proposal or an implementation plan that outlines a way to solve the problem or complete the task.
- [ ] This issue requires assistance or support from other groups, and it's indicated in the issue description.
- [ ] This issue could affect application security or performance, and the concern is explained in the issue description.
- [ ] This issue is the smallest iteration possible and doesn't require further break down.
-] This issue has weight set - according to [this list of possible values - and ~"needs weight" label is removed.
- [ ] This issue has a success criteria defined, and it is outlined in the issue description.
- [ ] This issue is labeled correctly.
- [ ] This issue is reviewed by another team member to confirm proposal/implementation plan and weight.
- [ ] Finally, add ~"workflow::ready for development" label to this issue and unassign yourself.

Refinment Review Guidelines

If you're assigned this issue to review its refinement, please follow the guidelines below.

1. Please validate the proposal or the implementation plan described in the issue.
1. Please validate the weight of the issue according to this list of possible values.
1. If in disagreement, please state your thoughts/reasoning and notify the engineer refining this issue.
1. If the disagreement can't be resolved, please bring this issue to the next team meeting for discussion.

Issue Assignmet

Issues are assigned randomly to engineers using triage-ops bot. The process works like follows:

1. Planning issue is created, and a number of issues are selected for the next milestone (marked with labels defined below).

1. Issues selected are labeled with:
    1. ~"group::secret detection"
    1. ~"workflow::planning breakdown"
1. A scheduled policy is triggered monthly before the upcoming milestone begins (on 2nd Thursday of a month, or exactly one week before the new milestone starts).
1. The scheduled operation runs and does the following:
    1. Pick up issues with the following conditions:
        1. State: Opened
        1. Labels:
            1. ~"group::secret detection"
            1. ~"workflow::planning breakdown"
        1. Weight:
            1. None.
        1. Milestone:
            1. Issue has a milestone.
            1. Issue's milestone title = nextmilestone.
    1. Actions:
        1. Assigns the issue to a random engineer from the Secure:Secret Detection group.
        1. Adds the following labels:
            1. ~"worfklow::refinement"
            1. ~"needs weight"
        1. Removes the following labels:
            1. ~"worfklow::planning breakdown"
        1. Adds the comment below.

Comment

markdown
Hi {secretdetectionengineer}

As a preparation for the upcoming milestone {milestone.succ}, you have been assigned this issue to refine.

The goal of the process is to:

- Clarify any outstanding questions or concerns.
- Add a proposal or an implementation plan.
- Determine if the issue is the smallest iteration possible, and break it down if not.
- Determine if the issue requires support from other teams.
- Assign a weight to the issue.
- Ensure the issue is labeled correctly.
- Ensure issue is marked as ready to be worked on.

Please check the steps to follow and the checklist to use for keeping refinement progress transparent.

If you have any questions, don't hesitate to ask in gsecuresecret-detection channel.

Bot policy.

/assign {secretdetectionengineer}

Unplanned work

In general, the Secret Detection group has two sources of unplanned work: community contributions and ~severity::1 bugs. We will reserve capacity each release so we can respond quickly and efficiently. In both scenarios, we will route community contributions to the engineer who "owns" the analyzer.

We do, however, own and contribute to projects beyond the analyzers shipped as part of GitLab's product. Where possible, unplanned work requiring the attention of an engineer in Secret Detection will be routed according to that project's CODEOWNERS file. Otherwise, unplanned work will be considered and handled on a case-by-base basis.

Support to customers and prospects

While we plan our work on a monthly basis, customers and customer-facing team members may need support on an unplanned basis.
We aim to support these requests quickly because they affect the success of our customers and our business.

Generally, we aim to provide an initial response and triage the question/report as quickly as is reasonable.
"Reasonable" means, for example, that team members are answering during their normal working hours and are continuing their normal work activities.
Whoever is available and can contribute to a solution is encouraged to make first contact with the questioner and ask any clarifying questions·remember, you can always tag in another group member later if you're unable to resolve the question.

The aim of the triage is to support other team members in moving forward; if development work is required to address the problem, it is not automatically a top priority for the group and should not automatically displace existing planned work.
If there is any question of whether a bug fix or improvement should be taken up immediately, the Engineering Manager and Product Manager should be alerted to facilitate a decision.

Observability

For GitLab.com, we monitor performance of our code within the Rails application, metrics around our CI build performance, and traffic to our container registries. These dashboards are accessible on the Monitoring page.

Secure::Secret Detection Group Error Budget

Metrics

The process to add metrics to our projects is documented in our Metrics page.

Runbooks

The process for monitoring, responding to, and mitigating incidents is documented within our Secret Detection Runbooks page.

SECTION: engineering/development/sec/secure/secret-detection/metrics/_index.md

Overview

This page documents the process a member of the Secure: Secret Detection team should use to add metrics to capture product insights for the features we develop.

You should this guide to help you understand the 2 different types of metrics we can use, when to use each one, and give you a jumpstart in implementing them.

Metrics workflow

In general, the workflow for developing and adding metrics is:

mermaid
flowchart TD
    A[Product and team determine desired insights] --> B(Implement metrics)
    B --> C[Verify metrics are visible in Tableau]
    C -->D[PDI team helps with creating visualizations]

Creating metrics

For our use cases, we utilize Internal Tracking Events or Database Metrics (formerly Service Ping) depending on the situation:

- Internal Tracking Events are good for capturing events that occur but aren't stored in the database. E.g., a user clicks a specific button.
- Database metrics are useful when the data you want is stored in the database in some way that can be extracted with the right query. E.g., the nubmer of projects that have a setting enabled.

The Analytics Instrumentation team has great documentation here but we outline some of the learnings on implementing metrics for our use here.

Internal tracking events

Internal Tracking Events capture discrete events and can be collected over 7 days, 28 days, or all time. These events require code changes to explicitly fire the event.

Here is an example from Notes::BaseService

ruby
...

include Gitlab::InternalEventsTracking

...

trackinternalevent('createcommitnote', project: project, user: currentuser)
...

Each event has a descriptive name and, when possible, useful data for context like:

- user
- project
- namespace (which will be pulled from project if not supplied)

There's another field, category that is automatically set to the classname of
the class that the event was fired from. This is important to know for testing.

Each event can also have up to 3 additional data. Events support 2 string values, and 1
numeric.
These additional properties are stored in the additionalproperties map in the
event and the keys are:

- label (string)
- property (string)
- value (numeric)

You should utilize them in that order when possible, i.e., label before
property.

Process for adding

You should follow the process Analytics Instrumentation has defined in the quick start guide.
linked docs.

TL;DR Run the ruby scripts/internalevents/cli.rb CLI tool and follow the
prompts. The event definition is necessary for the code to know what
events to output and the metric definition is what Snowflake and Tableau will
make available given the events.

Testing

There are shared examples that you can utilize to test firing of tracking
events:

ruby
itbehaveslike 'internal event tracking' do
  let(:event) { "detectsecrettypeonpush" }

```
    let(:namespace) { project.namespace }
    let(:label) { "GitLab Personal Access Token" }
    let(:category) { describedclass.name }

    ...
    subject
end
```

Unlike in the implementation file, when using the shared example in the specs, you will need to define the :category to be the class under test.

Shared Examples

If you're adding internal event tracking tests to shared specs, you need to be able to redefine the subject to be what triggers the event firing if it isn't already.

As an example, in the specs for Gitlab::Checks::SecretsCheck we use the shared examples from secretschecksharedexample.rb.

In that file, most of the specs call subject.validate! to run the secrets check, but for the internal tracking shared examples, it expects to be able to just call subject.

Therefore, to use the internal tracking event shared examples from our shared examples we have to redefine subject to `subject { super().validate! }. super() within the subject{}` block refers to the predefined subject object, i.e., the Gitlab::Checks::SecretsCheck class.

So in this special case, we add a shared example internal event tracking to be:

```
ruby
itbehaveslike 'internal event tracking' do
    let(:event) { 'skipsecretpushprotection' }
    let(:namespace) { project.namespace }
    let(:label) { "commit message" }
    let(:category) { describedclass.name }
    subject { super().validate! }
  end
```

Database Metric (Service Ping)

Database metrics, aka Service Pings, are metrics that can be collected with database queries. These metrics are updated in a batch approximately every 7

days. However, this is not guaranteed and may be generated anywhere from 4-10 days.

Process for adding

Database metrics are implemented by a Ruby subclass of
GitLab::Usage::Metrics::Instrumentation::DatabaseMetric and
utilizes ActiveRecord relations to build the queries. Alternatively, you can
provide the SQL for the query too.

The class should be in lib/gitlab/usage/metrics/instrumentation/ or the EE
equivalent.

We have a Rails
generator
that can be used to create the necessary classes:

ruby
rails generate gitlab:usagemetric CountIssues --type database --operation distinctcount
create lib/gitlab/usage/metrics/instrumentations/countissuesmetric.rb
create spec/lib/gitlab/usage/metrics/instrumentations/countissuesmetricspec.rb

The simplest way to implement the metric is to call the class-level operation and relation
methods.

The argument to operation can be

- :count
- :distinctcount
- :estimatebatchdistinctcount
- :sum
- :average

relation takes a block that returns the query results.

Example from
`Gitlab::usage::Metrics::Instrumentation::CountProjectsWithSecretPushProtectionEnabledMetric:

ruby
```
class CountProjectsWithSecretPushProtectionEnabledMetric < DatabaseMetric
  operation :count

  relation do
    ProjectSecuritySetting.where(prereceivesecretdetectionenabled: true)
  end
end
```

Each database metric has to have an accompanying metric dictionary like Internal

Tracking Events. Unfortunately, database metrics are not yet supported by the internalevents CLI script so must be partially done by hand.

1. Create a yaml file in the appropriate subdirectory of config/metrics or ee/config/metrics if it's a metric limited to an enterprise tier.
    1. If the metric is meant to capture all time, use the countsall subdirectory.
    1. Otherwise use the appropriate counts7d or counts28d subdirectory for weekly and monthly metrics respectively.
1. Use existing yaml files as templates
1. Use the schema defined here.

NOTE: Make sure that the milestone is a string.

Testing

Like Internal Tracking Events, database metrics have shared examples that we can utilize in our tests.

ruby
itbehaveslike 'a correct instrumented metric value', { timeframe: 'all', datasource: 'database' }

timeframe should match the value in the dictionary for the metric that was defined, i.e., 7d, 28d, or all.

Viewing and analyzing

Verifying creation and deployment

Metrics are collected into Snowflake which are then viewable in Tableau. To verify a metric is in production and being generated there are 2 locations to check:

1. Metrics dictionary
1. Tableau Service Ping Exploration (You need Explorer level access or higher to Tableau)

The Metrics dictionary only shows what metrics are available and gives you the ability to copy the Snowflake query to get its values.

The Tableau Service Ping explorer shows the basic values of and allows you see the last 5 generated values. Further analysis must be done either in Tableau or Snowflake.

With the Explorer role in Tableau, you will be able to create dashboards but will be limited to using data sources created by someone with higher permissions. Any new Internal Tracking Event or Database Metric should be included in existing data sources

- Mart Ping Instance Metric Monthly
- Mart Ping Instance Metric Weekly

Asking for help

If you're not familiar with Tableau, creating worksheets and dashboards in it, and haven't worked through the Tableau-hosted courses, you have some options for help:

1. For help in creating Tableau dashboards and visualizations, the Product Data Insights (PDI) team has an issue intake process where you can request their help.
1. For specific questions on your Tableau worksheet or dashboard, you can reach out to the PDI team on their slack channels:
    - data-tableau for Tableau-specific help
    - data for any data-related question

Troubleshooting

If the metrics don't show up in Tableau or Snowplow you should contact gmonitoranalyticsinstrumention or datatableau slack channel.

If, in Tableau, you can't find either of the 2 data sources mentioned above, make sure to use the New Data Source button, then click the See All link on the right side above the table of available data sources.

SECTION: engineering/development/sec/secure/secret-detection/runbooks/_index.md

Overview

This page lists runbooks used by the Secret Detection team for monitoring, mitigating and responding to an incident.

Runbooks

Secret Push Protection feature
  Monitoring
  Troubleshooting
  Performance Testing
Secret Detection Service
  General FAQs
  Monitoring
  Troubleshooting (TBA, once we complete 494910 and 499249)

SECTION: engineering/development/sec/secure/secret-detection/runbooks/secret-detection-svc-faqs.md

This page contains answers to the general questions about the Secret Detection Service. This runbook can be used by anyone who want to understand the technical aspects about the service.

General FAQs

1. Where is the service deployed?

   The service is deployed on Runway which internally uses Google Cloud Run to manage containers.

2. In how many environments will the service be deployed?

   The service is deployed in Staging (https://secret-detection.staging.runway.gitlab.net) and Production (https://secret-detection.production.runway.gitlab.net).

3. In what regions will the service be deployed in the production environment?

   The service is deployed only at us-east1, the same region where the GitLab Rails monolith is deployed too.

4. Does the service use environment variables?

   Yes. Currently, it uses the ENV(non-sensitive) var to determine the active environment the application is running and the AUTHTOKEN(sensitive) var to match it against the token embedded in the API request.

5. Where are environment variables stored in the service and who has access to modify them?

   Non-sensitive variables are stored in env-<environment>.yml files in the project repository whereas sensitive variables are stored in the Hashicorp Vault. Currently, the ~"group::secret detection" team has access to the vault.

6. Where exactly in the vault do I find SD service variables?

   You can find them here for the staging environment, and here for the production environment.

7. Are the service APIs publicly accessible?

   It depends on the environment. Currently, the service in the staging environment is publicly accessible and we might change it to private later. To reduce security exposure, the service in the production environment is made accessible only by the GitLab Rails monolith instance.

8. What category of APIs does the service expose?

   The service exposes only gRPC endpoints for Secret Detection scans. Read here for more details.

9. Is there an authentication process to access service APIs?

Yes. A basic form of token-based authentication where the client is expected to embed a token in the request which is matched against the AUTHTOKEN in the service. Note that the authentication is only applied to secret detection-related RPC endpoints. Read more about it here.

10. How do I ensure that the service is running? Is there a health-check endpoint to confirm?

The service exposes a health check RPC endpoint used by Runway to ensure the service's health. You can find it here. However, for the production environment, we might have to rely on logs for health check failures since the instance isn't accessible publicly. Alternatively, we could access it from a teleport console since Rails monolith can access the service.

11. Which GitLab services access SD service?

Only the Rails monolith service accesses the SD service to invoke scans.

12. Where can I access the service logs?

We can access them through Google Cloud Run logs. We can view logs through GCP Logs Explorer too to get custom filtering/querying abilities.

13. Where can I access the service dashboard for monitoring purposes?

You can find them here for the staging environment and here for the production environment.

14. Is there any rate limiting added to the APIS?

Application-level rate limiting is not added, however, Cloud Run defines rate limiting for the instances under which the service is covered.

15. What are the Service Level Indicators(SLIs) for the service to determine the availability?

We are using Runway's default SLIs(runwayingress) which contains the Apdex Score, Request Rate, and Error Rate.

16. What are the Service Level Objectives(SLOs) configured for the service?

SLOs for the Service are set to meet 99.9% (0.999) of the Apdex Score and 99.9% of (0.999) Error Ratio. These are default SLO values configured by Runway and we are sticking to the default values as they seemed sufficient for the service. We can change it here whenever necessary.

17. Do we have alerts configured in case there is an SLO violation?

Yes. The alerts are configured here. The alerts are triggered for Apdex violations, Error Rate violations, Traffic ceased (Server signal present, traffic none), and Traffic absent (No server signal, including Health Checks) in the past 30 minutes.

18. What happens in the event of an SLO violation?

In case of an SLO violation incident, the alertmanager fires all alerts to the feedalerts-general slack channel, and also a copy of it will be sent to gsecure-secret-detection slack channel.

19. What severity will be assigned to the triggered alert?

Since the service borrows Runway's SLI defaults, the defaults also include setting severity as S4. We can change it to a different severity appropriate to our needs (requires Readiness Review approval).

20. Does it page the SRE team in the event of an SLO violation?

No. Only the alerts with severity S1 or S2 are paged to the SRE team. The group::secret detection team will be responsible for monitoring incidents.

Additional References

Documentation

Architecture

Benchmarks

Monitoring

SECTION: engineering/development/sec/secure/secret-detection/runbooks/secret-detection-svc-monitoring.md

When to use this runbook?

This runbook is intended to be used when monitoring the Secret Detection Service to identify and mitigate any reliability issues or performance regressions that may occur when it is enabled on Gitlab.com and/or Dedicated.

What to monitor?

We primarily need to monitor system metrics and recurrent errors raised within the service. Here are the narrowed down list of monitoring targets:

Resource Saturation: Saturation is a measure of what ratio of a finite resource is currently being utilized.
Aggregated Service Level Indicators(SLIs)
Apdex Score: Apdex is a measure of requests that complete within a tolerable period of time for the service.
Error Ratio: Error rates are a measure of unhandled service exceptions per second. Client errors are excluded when possible.
Request Rate: The operation rate is the sum total of all requests being handle for all components within this service. Note that a single user request can lead to requests to

multiple components.
Recurrent appplication errors raised by the service.

How to monitor the service?

Most of above-mentioned monitoring targets i.e. Resource Saturation and Aggregated SLIs, are available in the Service Overview Dashboard.

The recurrent application errors are generally available in the GitLab Error Monitoring/Sentry tool. However, we are yet to integrate the service with the Error Monitoring tool. Please refer to this issue to track the integration progress.

Meanwhile, you may refer to the logs for any log messages with ERROR levels to look for application-related errors. Logs are available here.

How to know if there is an SLO violation for the service?

The service receives slack alerts in the event of SLO violation. Currently, six alerts are configured by the alertmanager tracking multiple metrics.

In the event of SLO violation, the alertmanager sends an alert to feedalerts-general and gsecure-secret-detection Slack channels.

Note: The alertmanager does not page on-call SRE team member in the event of SLO violation for this service. The service inherits Runway's default SLIs which sets the alert severity to S4. Only the alerts marked as S1 or S2 will page the on-call. In other words, Secure::Secret Detection team will be responsible for addressing service disruption incidents by keeping an eye on the slack alerts.

In case the alertmanager failed to trigger a slack alert, we can always find the alerts that are actively being fired for the service in the alerts dashboard.

How to check the logs emitted by the service?

Runway uses GCP Cloud Logs to manage logs emitted by the service. The logs for the service are available here.

Additional References

Documentation

Architecture

Service FAQs

Runway Monitoring Stack

SECTION: engineering/development/sec/secure/secret-detection/runbooks/secret-push-

protection-monitoring.md

When to use this runbook?

This runbook is intended to be used when monitoring the secret push protection feature to identify and mitigate any reliability issues or performance regressions that may occur when it is enabled on Gitlab.com. The runbook can also be used to understand more about relevant dashboards below and how to improve them:

Secret Push Protection · Overview Dashboard

What to monitor?

While the feature, in its current form, doesn't have any external components and is entirely encapsulated within the application server as a dependency, it does interact with a number of components as can be seen in this push event sequence diagram. Those components are:

GitLab Shell (Git over SSH):
  git-receive-pack
Workhorse (Git over HTTP/S):
  git-receive-pack
Gitaly:
  SSHReceivePack
  PostReceivePack
  PreReceiveHook
  ListAllBlobs() RPC
  ListBlobs() RPC
  GetTreeEntries() RPC
Rails:
  /internal/allowed Endpoint

Below is a sequence diagram showing the entire workflow whether a git push takes place over HTTP or SSH:

mermaid
sequenceDiagram
    actor User
    User->>+Workhorse/GitLab Shell: git push
    Workhorse/GitLab Shell->>+Gitaly: tcp/ssh
    Gitaly->>+Gitaly: PostReceivePack/SSHReceivePack
    Gitaly->>+Gitaly: git-receive-pack
    Gitaly->>+Gitaly: PreReceiveHook
    Gitaly->>+Rails: grpc
    Note over Gitaly, Rails: invokes /internal/allowed endpoint
    Rails->>+Rails: GitLab::GitAccess
    Rails->>+Rails: EE::GitLab::Checks::ChangesAccess
    Note over Rails: runs Gitlab::Checks::SecretsCheck
    break when special commit message flag is found
        Rails->>+Gitaly: push check skipped
        Gitaly->>+Workhorse/GitLab Shell: outcome of push

```
    Workhorse/GitLab Shell->>+User: outcome of push
  end
  break when push option is passed
    Rails->>+Gitaly: push check skipped
    Gitaly->>+Workhorse/GitLab Shell: outcome of push
    Workhorse/GitLab Shell->>+User: outcome of push
  end
  Rails->>+Gitaly: ListBlobs or ListAllBlobs
  Note over Gitaly, Rails: depends on quarantine directory existence
  Gitaly->>+Rails: grpc
  Rails->>+gitlab-secretdetection: gitlab-secretdetection::Scan
  alt no secret detected
    gitlab-secretdetection->>+gitlab-secretdetection: scan blob
    gitlab-secretdetection->>+Rails: success
    Rails->>+Gitaly: accept - no secret detected
  else scan timeout
    gitlab-secretdetection->>+gitlab-secretdetection: scan blob
    gitlab-secretdetection->>+Rails: fail - timeout
    Rails->>+Gitaly: accept - scan timeout
  else secret detected
    gitlab-secretdetection->>+gitlab-secretdetection: scan blob
    gitlab-secretdetection->>+Rails: fail - secret found
    Rails->>+Gitaly: GetTreeEntries
    Note over Gitaly, Rails: retrieves blobs' file path and commit sha
    Gitaly->>+Rails: grpc
    Rails->>+Rails: Format Response
    Rails->>+Gitaly: reject - secret detected
  end
  Gitaly->>+Workhorse/GitLab Shell: outcome of push
  Workhorse/GitLab Shell->>+User: outcome of push
```

Note: PreReceiveHook is not to be confused with git's pre-receive hook. In fact, the former is a binary wrapper around the actual git hook. Please read more about the hook setup in Gitaly's documentation.

These components are therefore the main elements we are trying to focus on when monitoring the feature.

How we monitor the feature?

As discussed above, the functionality spans a number of components. Therefore, are three main tools we could use for monitoring the feature:

- Kibana (Logs)
  - Staging
    - pubsub-rails-inf-gstg
    - pubsub-gitaly-inf-gstg
    - pubsub-workhorse-inf-gstg
    - pubsub-shell-inf-gstg

Production
pubsub-rails-inf-gprd
pubsub-gitaly-inf-gprd
pubsub-workhorse-inf-gprd
pubsub-shell-inf-gprd
Prometheus/Grafana (Metrics)
Internal API
Gitaly
GitLab Shell
Sentry (Error Tracking)
Gitlab.com
Gitaly
Workhorse

This runbook focuses primarly on the Prometheus metrics available in Grafana, but also shares brief information about other tools and how they could be used. In later iterations, this may change as the feature grows and develops.

How to check the logs emitted from the feature?

To check the logs emitted from the feature, please look at the following Kibana views:

· Kibana: Logs for all scans.
· Kibana: Logs for blocked pushes.

>Note: Kibana retain logs for only 7 days.

How to identify and mitigate a reliability or performance issue with the feature?

The overview dashboard is the main dashboard we have built to monitor the feature. That's where anyone should start to look when trying to identify reliability or performance issues.

The dashboard itself is split into 4 rows (or sections), with each containing a number of panels as below.

GitLab Shell (Git over SSH)

This section monitors the stability of certain operations related to the feature within Gitlab Shell, which is a set of executables created to handle Git SSH sessions. The tool itself does not handle SSH directly, but instead the SSH server/daemon gitlab-sshd maintain all connections with clients and calls up Rails via GitLab Shell to perform authorization or access checks. Please check this diagram and this description of a request cycle for more information on how that works.

The section can be used to ensure there are no performance degradations related to git-receive-pack operations when a git push operation is carried out over SSH. It is dividend into two rows/sections as follows.

Note: Most of available metrics for both gitlab-shell and gitlab-sshd aren't aggregated by the command used, so for a more better overview of the performance of git-receive-pack operation,

take a look at the Kibana logs linked in those sections instead.

gitlab-shell

RPS (Requests Per Seconds)

This panel displays average number of requests per second (RPS) made to gitlab-shell over time. The panel can be used to monitor request rates and understand if there's a performance or scalability issue. Use the link for more detailed overview of this metric. Note: this isn't specific to git-receive-pack command.

Panel Information

Metric: gitlabcomponentops:rate5m
Label Filters:
  component = gitlabshell
  env = gprd
  monitor = global
  stage = main
  type = git
Operations:
  Avg over time: range | $interval
Legend:
  RPS

Total Established Gitaly Connections

This panel displays the total number of Gitaly connections that have been established by gitlab-shell at a given time. This panel can be used to determine if there's a sudden drop in connections between both components, which may indicate a performance or an availability issue. Note: this isn't specific to git-receive-pack command.

Panel Information

Metric: gitlabshellgitalyconnectionstotal
Label Filters:
  env = gprd
  stage = main
Operations:
  Count:
    Label: status
Legend:
  Auto

Established SSH Sessions

This panel displays the minimum number of established SSH sessions at a given time. The panel can be used to understand if there's an availability issue together with the panel adjacent to it which shows how the maximum number of SSH sessions that failed to establish at a given time. Note: this isn't specific to git-receive-pack command.

Panel Information

Metric: gitlabsli:shellsshdsessions:total
Label Filters:
env = gprd
stage = main
type = git
Operations:
Min
Legend:
Auto

Failed SSH Sessions

This panel displays the maximum number of failed SSH sessions at a given time. The panel can be used to understand if there's an availability issues together with the panel adjacent to it which shows how the minimum number of SSH sessions established over a given time. Note: this isn't specific to git-receive-pack command.

Panel Information

Metric: gitlabsli:shellsshdsessions:errorstotal
Label Filters:
env = gprd
stage = main
type = git
Operations:
Max by:
Label: app
Legend:
Auto

Established Session Average Duration

This panel displays the average duration of establish SSH sessions summed up over a range of 24 hours. The panel can be used to determine if there's an increase in the duration of a git pull/push over SSH which may indicate a performance or availability issue. Note: this isn't specific to git-receive-pack command.

Panel Information

Metrics:
gitlabshellsshdsessionestablisheddurationsecondssum
gitlabshellsshdsessionestablisheddurationsecondscount
Label Filters:
env = gprd
stage = main
type = git
Operations:
Divison: /

Rate: range | 24h
Sum
Legend:
{{labelname}}

All Sessions Average Duration

This panel displays the average duration of all SSH sessions (whether established or failed) summed up over a range of 24 hours. The panel can be used to determine if there's an increase in the duration of a git pull/push over SSH which may indicate a performance or availability issue. Note: this isn't specific to git-receive-pack command.

Panel Information

Metrics:
gitlabshellsshdsessiondurationsecondssum
gitlabshellsshdsessiondurationsecondscount
Label Filters:
env = gprd
stage = main
type = git
Operations:
Divison: /
Rate: range | 24h
Sum
Legend:
{{labelname}}

gitlab-sshd

SLI Apdex

This panel displays the application performance index (Apdex) for the gitlab-sshd SSH daemon/server. This Service Level Indicator (SLI) averages close to 99.9%, most of the time but a drop in the indicator could point to an outage or degradation. Use the link for more detailed overview of this metric. Note: this isn't specific to git-receive-pack command.

Panel Information

Metric: gitlabcomponentapdex:ratio5m
Label Filters:
component = gitlabsshd
env = gprd
monitor = global
stage = main
type = git
Operations:
Min over time: range | $interval
Legend:
Apdex

SLI Error Ratio

This panel displays the max ratio of errors clamped to maximum value received for the gitlab-sshd SSH daemon/server. This Service Level Indicator (SLI) averages close to 0.01%, most of the time but an increase in the indicator could point to an outage or degradation. Use the link for more detailed overview of this metric. Note: this isn't specific to git-receive-pack command.

Panel Information

Metric: gitlabcomponenterrors:ratio5m
Label Filters:
component = gitlabsshd
env = gprd
monitor = global
stage = main
type = git
Operations:
Clamp max:
Maximum Scalar: 1
Max over time: range | $interval
Legend:
Error %

SLI RPS (Requests Per Second)

This panel displays average number of requests per second (RPS) made to gitlab-sshd SSH daemon/server over time. The panel can be used to monitor request rates and understand if there's a performance or scalability issue. Use the link for more detailed overview of this metric. Note: this isn't specific to git-receive-pack command.

Metric: gitlabcomponentops:ratio5m
Label Filters:
component = gitlabsshd
env = gprd
monitor = global
stage = main
type = git
Operations:
Avg over time: range | $interval
Legend:
Error %

Workhorse (Git over HTTP/S)

This section monitors the stability of certain operations related to the feature within Workhorse, which is a smart reverse proxy intended to handle resource-intensive and long-running requests. It intercepts all HTTP requests and either propagates them without changing or handles them itself by performing additional logic. Please check this diagram and this description of a request cycle for more information on how that works.

The section can be used to ensure there are no performance degradations related to git-receive-pack operations when a git push operation is carried out over HTTP/S.

Processed /.git/git-receive-pack Requests

This panel displays the number of HTTP requests that have been processed by workhorse over time, increasing in range of 24 hours. The panel partitions these requests by the HTTP verb/method and response code. This panel can be used to determine if the amount of git-receive-pack requests with a response code that isn't 200 had increased recently, indicating an issue with processing such requests.

Panel Information

```
 Metric: gitlabworkhorsegithttprequests
 Label Filters:
   exportedservice = git-receive-pack
   env = gprd
   stage = main
   code != 0
 Operations:
   Increase: range | 24h
   Sum:
     Label: code
     Label: method
 Legend:
   {{code}} | {{method}}
```

Total Established Gitaly Connections

This panel displays the total number of Gitaly connections that have been established by workhorse at a given time. This panel can be used to determine if there's a sudden drop in connections between both components, which may indicate a performance or an availability issue.

Panel Information

```
 Metric: gitlabworkhorsegitalyconnectionstotal
 Label Filters:
   env = gprd
   stage = main
 Operations:
   Count:
     Label: status
 Legend:
   {{status}}
```

Average Latency for /.git/git-receive-pack Request [All Nodes]

This panel displays the average latency (duration) in seconds for the /.git/git-receive-pack request for all nodes running workhorse. This panel can be used to determine if there is an increase in response times for that specific request, which could indicate performance

degradation issue if it surpassed a certain thershold.

Panel Information

Metrics:
gitlabworkhorsehttprequestdurationsecondssum
gitlabworkhorsehttprequestdurationsecondscount
Label Filters:
env = gprd
stage = main
route = ^/.+\\.git/git-receive-pack\\z (double escaping is used for backslash)
Operations:
Divison: /
Rate: range | 24h
Sum:
Label: node
Legend:
Auto

SLI Apdex

This panel displays the application performance index (Apdex) for the workhorse component. This Service Level Indicator (SLI) averages close to 99.9%, most of the time but a drop in the indicator could point to an outage or degradation. Use the link for more detailed overview of this metric. Note: this isn't specific to /.git/git-receive-pack route.

Panel Information

Metric: gitlabcomponentapdex:ratio5m
Label Filters:
component = workhorse
env = gprd
monitor = global
stage = main
type = git
Operations:
Min over time: range | $interval
Legend:
Apdex

SLI Error Ratio

This panel displays the max ratio of errors clamped to maximum value received for workhorse. This Service Level Indicator (SLI) averages close to 0.001%, most of the time but an increase in the indicator could point to an outage or degradation. Use the link for more detailed overview of this metric. Note: this isn't specific to /.git/git-receive-pack route.

Panel Information

Metric: gitlabcomponenterrors:ratio5m

Label Filters:
component = workhorse
env = gprd
monitor = global
stage = main
type = git
Operations:
Clamp max:
Maximum Scalar: 1
Max over time: range | $interval
Legend:
Error %

SLI RPS (Requests Per Second)

This panel displays average number of requests per second (RPS) made to workhorse over time. The panel can be used to monitor request rates and understand if there's a performance or scalability issue. Use the link for more detailed overview of this metric. Note: this isn't specific to /.git/git-receive-pack route.

Metric: gitlabcomponentops:ratio5m
Label Filters:
component = workhorse
env = gprd
monitor = global
stage = main
type = git
Operations:
Avg over time: range | $interval
Legend:
Error %

Gitaly

This section monitors the stability of gitaly, a tool providing high-level RPC access to git repositories. We focus in this section on the hooks and RPCs used by the feature. This can be used to ensure there are no performance degradations related any of them.

The section is divided into four sub-sections as follows, with most focus being on latency.

1. GitLab Shell <=> Gitaly:
    SSHReceivePack
1. Workhorse <=> Gitaly:
    PostReceivePack.
1. Gitaly <=> Rails API:
    Gitaly / Before /internal/allowed:
        PreReceiveHook.
    Gitaly / During /internal/allowed:
        ListAllBlobs() RPC
        ListBlobs() RPC

GetTreeEntries() RPC

GitLab Shell <=> Gitaly

PostReceivePack · Average Latency [All Hosts]

This panel displays the average latency on all hosts in milliseconds for calls to the PostReceivePack RPC, which is the RPC responsible for calling git-receive-pack command, that in turn executes the PreReceiveHook. The latter goes on to call /internal/allowed endpoint that runs access checks on Rails side.

Panel Information

Metric: gitaly:grpcserverhandlingseconds:avg5m
Label Filters:
job = gitaly
grpcmethod = PostReceivePack
Operations:
Avg: 1000 avg
Legend:
{{method}}

Workhorse <=> Gitaly

SSHReceivePack · Average Latency [All Hosts]

This panel displays the average latency on all hosts in milliseconds for calls to the SSHReceivePack RPC, which is the RPC responsible for calling git-receive-pack command (for git push over SSH), that in turn executes the PreReceiveHook. The latter goes on to call /internal/allowed endpoint which runs access checks on Rails side.

Panel Information

Metric: gitaly:grpcserverhandlingseconds:avg5m
Label Filters:
job = gitaly
grpcmethod = SSHReceivePack
Operations:
Avg: 1000 avg
Legend:
{{method}}

Gitaly <=> Rails API

Gitaly / Before /internal/allowed

PreReceiveHook · Average Latency [All Hosts]

This panel displays the average latency on all hosts in milliseconds for calls to the PreReceiveHook hook, that in turn calls /internal/allowed endpoint to runs access checks on

Rails side.

Panel Information

Metric: gitaly:grpcserverhandlingseconds:avg5m
Label Filters:
job = gitaly
grpcmethod = PreReceiveHook
Operations:
Avg: 1000 avg
Legend:
{{method}}

Gitaly / During /internal/allowed

ListAllBlobs · Average Latency [All Hosts]

This panel displays the average latency in milliseconds for all calls to the ListAllBlobs RPC, which is responsible (within the context of the feature) for enumerating all blobs of a repository under a certain size limit (i.e. exactly 1MiB). This procedure is usually fast because it is mostly used with the size limit set to 0 for checking file sizes of blobs in a certain git push.

Panel Information

Metric: gitaly:grpcserverhandlingseconds:avg5m
Label Filters:
job = gitaly
grpcmethod = ListAllBlobs
Operations:
Avg: 1000 avg
Legend:
{{method}}

ListBlobs · Average Latency [All Hosts]

This panel displays the average latency in milliseconds for all calls to the ListBlobs RPC, which is responsible (within the context of the feature) for enumerating all blobs of a repository under a certain size limit (i.e. exactly 1MiB), similar to ListAllBlobs, but it also loads up file paths for those blobs. The procedure is often slower than ListAllBlobs because it loads up blob contents when enumerating them.

Panel Information

Metric: gitaly:grpcserverhandlingseconds:avg5m
Label Filters:
job = gitaly
grpcmethod = ListBlobs
Operations:
Avg: 1000 avg

Legend:
{{method}}

GetTreeEntries · Average Latency [All Hosts]

This panel displays the average latency in milliseconds for all calls to the GetTreeEntries RPC, which is responsible (within the context of the feature) for retrieving blob metadata (i.e. file path and commit sha) for all blobs that were scanned and found to include a leaked secret.

Panel Information

Metric: gitaly:grpcserverhandlingseconds:avg5m
Label Filters:
job = gitaly
grpcmethod = GetTreeEntries
Operations:
Avg: 1000 avg
Legend:
{{method}}

Rails

This section monitors the stability of the /internal/allowed endpoint which is a focal point in the feature's journey to protect against leaked secrets in a git push. The endpoint is part of GitLab's Internal API, and is responsible for assessing if a user has permission to perform certain operations on the repository.

The section can be used to ensure there are no performance degradations related to the /internal/allowed endpoint when changes in a certain git push are scanned for secrets.

Internal API / Request Latency

This panel displays the average, p95, p99, and mean latencies for requests made to the /internal/allowed endpoint over time. The panel can be used to monitor request latency and understand if there's a performance or scalability issue with that specific endpoint. Use the link for more detailed overview of this metric.

Panel Information

Metrics:
controlleraction:gitlabtransactiondurationsecondssum:rate5m
controlleraction:gitlabtransactiondurationseconds:p95
controlleraction:gitlabtransactiondurationseconds:p99
controlleraction:gitlabtransactiondurationsecondssum:rate1m
controlleraction:gitlabtransactiondurationsecondscount:rate1m
Label Filters:
action = POST /api/internal/allowed
controller = Grape
environment = gprd

stage = main
type = internal-api
Operations:
Avg over time: range | $interval
Avg:
Label: controller
Label: action
Legends:
{{action}} · avg
{{action}} · p95
{{action}} · p99
{{action}} · mean

Internal API / RPS (Requests Per Second)

This panel displays average number of requests per second (RPS) made to the /internal/allowed endpoint over time. The panel can be used to monitor request rates and understand if there's a performance or scalability issue with that specific endpoint. Use the link for more detailed overview of this metric.

Panel Information

Metrics:
controlleraction:gitlabtransactiondurationsecondscount:rate1m
Label Filters:
action = POST /api/internal/allowed
controller = Grape
environment = gprd
stage = main
type = internal-api
Operations:
Avg over time: range | $interval
Sum:
Label: controller
Label: action
Legends:
{{action}}

Internal API / Memory Saturation Rate

This panel displays the memory saturation rate for two components of the internal API, which are Ruby VM and Puma Workers. This is helpful to understand if the memory consumption in Rails had increased to the point of saturation, which indicates a performance and a scalability issue and requires attention. Note: this panel isn't specific to /internal/allowed endpoint.

Panel Information

Metrics:
gitlabcomponentsaturation:ratio
Label Filters:

env = grpd
environemnt = gprd
stage = main
type = internal-api
component = rubythreadcontention | pumaworkers
Operations:
Max over time: range | $interval
Max:
Label: component
Legends:
Auto

Where else to look for help?

If you're unsure, you can always ask for help in gsecure-secret-detection channel.

How to improve this runbook?

The runbook needs to be updated as the feature evolves and progresses. Please follow guidelines below to keep it updated.

When a panel is updated in a dashboard

If a panel is updated in a dashboard, please update the panel information and description as needed.

When a new panel is added

If a new panel is created in a dashboard, please add the name, description, and information using the same format outlined below.

markdown
PANEL NAME IN BOLD

A few sentences describing what the panel does and what it could be used for to identify a performance regression or reliability issue.

Panel Information

Metric: NAMEOFMETRICUSED
Label Filters:
LISTOFLABELSUSEDTOFILTERBYINKEYANDVALUE
Operations:
LISTOFOPERATIONSAPPLIEDONDATA
Legend:
LEGENDUSEDIFNOTAUTOMATIC

When a panel is removed

In case a panel is removed from the dashboard, please consider removing the corresponding section from this runbook.

How to contribute to relevant dashboards?

Dashboards discussed in this runbook can be improved as follows.

When a new component is utilised by the feature

If a new component is utilised by the feature, please follow the steps below.

Identify endpoints or services the feature interacts with in the component.
Explore metrics available for the endpoint or service.
If no metrics are available, consider creating them to monitor the performance of the endpoint/service.
Create a new row for the component in the dashboard you are editing.
Add as many panels as for available metrics in the new row. Use your best judgement on what is should be added.
Create a merge request updating this runbook with information about the panel. Use panels above for guidance.

When a component is no longer relevant

If a component is no longer relevant, please remove its corresponding row from the dashboard.

SECTION: engineering/development/sec/secure/secret-detection/runbooks/secret-push-protection-performance-testing.md

When to use this runbook?

Use this runbook for:

Running GPT tests - for running tests and comparing with previous benchmarks
Deploying a new version of GitLab to GET - for updating a GET instance, most likely to test out changes related to secret push protection
Setting up a new GET environment - for testing different reference architectures

Prerequisites

gcloud (official instructions) - for running various commands, and for logging in to the test runner vm
The Static Analysis GCP Project (see Resources section) - access required to make changes to the infrastructure

Running GPT tests

Manual testing

Get the url and password for the root user from 1password by searching for Static Analysis in the Engineering Vault. Please don't delete projects, groups, or users, but feel free to create any of those, or anything else you'd like to test with.

Running automated tests (via GCP VM)

Options for logging in to the VM:

SSH in to the VM: gcloud compute ssh --zone us-west1-c gpt-test-runner-2 --project dev-sast-prereceive-8a4574ec
Or, go to The Static Analysis GCP Project: dev-sast-prereceive-8a4574ec, click Compute Engine, find gpt-test-runner-2, and click SSH. This will launch a web-based SSH session.

One time setup (required by everyone running the tests from the VM):

git clone https://gitlab.com/gitlab-org/quality/performance.git
cd in to performance
Copy the gcp-2k.json file from this MR to the performance directory
Create a .tool-versions file and add ruby 3.2.2 to it (or whatever the latest is)
bundle install
Check out our branch secret-detection

Running the tests:

Get the glpat token from 1password by searching for Static Analysis in the Engineering Vault
Ensure you are in the performance directory
To run just the gitsecretdetection.js test (as an example): ACCESSTOKEN=glpat-REDACTED SDPUSHCHECKENABLED=true SDFILESPERCOMMIT=4 GPTDEBUG=true SDFILESIZES="10kb" GPTS ./bin/run-k6 --environment gcp-2k.json --options 60s40rps.json --unsafe --tests k6/tests/git/pre-receive/gitsecretdetection.js
To run all the tests: ACCESSTOKEN=glpat-REDACTED SDPUSHCHECKENABLED=true ./bin/run-k6 --environment gcp-2k.json --options 60s40rps.json

Running automated tests (via Docker)

Get the glpat token (ACCESSTOKEN) from 1password by searching for Static Analysis in the Engineering Vault
GPT tests can be ran from any directory that contains an environments directory
Open a local terminal and git clone https://gitlab.com/gitlab-org/quality/performance.git
Create environments directory at the root
Copy the gcp-2k.json file from this MR to that environments directory
Rename the gcp-2k.json file as necassary, as well as changing the name and url values
To run just the gitsecretdetection.js test (as an example): docker run -it -e ACCESSTOKEN=glpat-REDACTED -v $PWD:/config gitlab/gitlab-performance-tool SDPUSHCHECKENABLED=true SDFILESPERCOMMIT=4 GPTDEBUG=true SDFILESIZES="10kb" GPTS --environment gcp-2k.json --options 60s40rps.json --tests gitsecretdetection.js
To run all the tests: docker run -it -e ACCESSTOKEN=glpat-REDACTED -v $PWD:/config gitlab/gitlab-performance-tool --environment gcp-2k.json --options 60s40rps.json

Setting up a GET

A GCP environment has been set up under The Static Analysis GCP Project: dev-sast-prereceive-8a4574ec with GET using a 2k reference architecture with a prefix of gcp-2k.

Bootstrapping a new environment

Note: The following steps are written from the perspective of setting up another 2